



WELCOME





Working with Strings & Input-Output Basics

Recap

Sequence of Instructions

- ✓ Sequence of Instructions
- ✓ BODMAS



Agenda for Today's Session

Working with Strings & Input Output Basics

Working with Strings

Type Conversion

Input Output Basics

Working with Strings

Adding Strings

Python

```
a = 1 + 2  
print(a)
```

Output

3

Python

```
a = "1" + "2"  
print(a)
```



What will be the output?

Adding Strings

Python

```
a = "1" + "2"  
print(a)
```

Output

12

- ✓ Anything within **quotes** (single or double) will be **treated as a string**
- ✓ **+ operator** is used between the **strings**, it **combines** (**concatenates**) them into a **single string**

String Concatenation

Example

Python

```
a = "Hello" + " " + "World"  
print(a)
```

Output

```
Hello World
```



String Concatenation

Example

JavaScript

```
a = "*" + 10  
console.log(a)
```

Output

*10

Python

```
a = "*" + 10  
print(a)
```



What will be the output ?

Concatenation Errors

Python

```
a = "*" + 10  
print(a)
```

Output

```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 1, in  
<module>  
TypeError: can only concatenate str  
(not "int") to str
```



Key Differences

Key Difference



Python

Requires **explicit conversion** of non-strings to **strings** for **concatenation**



JavaScript

Automatically converts non-strings to **strings** when **concatenating**

Explicit Conversion

- ✓ In **Python**, we can **convert** an **integer** to a **string** using the **str()** function
- ✓ This is an example of **explicit conversion**, also known as **type conversion**

Python

```
a = str(5)  
print(type(a))
```

Output

```
<class 'str'>
```

String Concatenation

Correct Code

Python

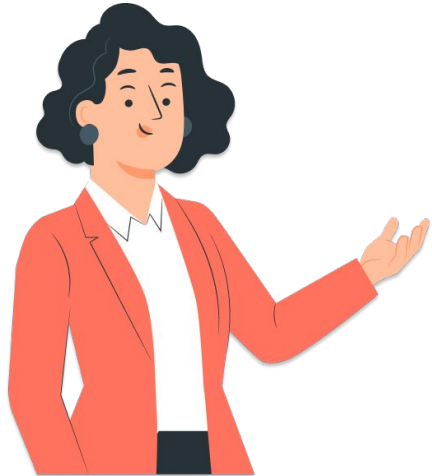
```
a = "*" + str(10)  
print(a)
```

Output

*10



Type Conversions



`int()`

Converts to **integer data type**

`float()`

Converts to **float data type**

`str()`

Converts to **string data type**

`bool()`

Converts to **boolean data type**

Type Conversion in Python

Conversion Type	Example	Output	Explanation
String → Integer	<pre>a = int("5") print(type(a))</pre>	<pre><class 'int'></pre>	Converts a valid numeric string to integer
Invalid String → Integer	<pre>a = int("Five")</pre>	<pre>ValueError: invalid literal for int()...</pre>	Raises ValueError if string is not numeric
String → Float	<pre>a = float("5.5") print(type(a))</pre>	<pre><class 'float'></pre>	Converts a valid numeric string to float
Invalid String → Float	<pre>a = float("abc")</pre>	<pre>ValueError: could not convert string to float</pre>	Raises ValueError if string is not a valid float

Type Conversion in Python

Conversion Type	Example	Output	Explanation
Boolean → Integer	<pre>b=int(True) print(b)</pre>	1	True → 1, False → 0
Integer → Boolean	<pre>b=bool(5) print(b)</pre>	True	Non-zero integers → True, 0 → False
Integer to String	<pre>a=str(123) print(a)</pre>	123	Converts a string to an integer
Integer to Float	<pre>a=float(3) print(a)</pre>	3.0	Converts integer to a float

Type Conversions

Python vs JavaScript

JavaScript	Python
<code>String()</code>	<code>str()</code>
<code>Number()</code>	<code>int()</code>
<code>parseInt()</code>	<code>int()</code>
<code>toString()</code>	<code>str()</code>

Type Conversions

Python vs JavaScript

JavaScript	Python
<code>parseFloat()</code>	<code>float()</code>
<code>Boolean()</code>	<code>bool()</code>

None to Integer Conversion

```
print(int(None))
```

Python

Output

```
Traceback (most recent call last):  
  File "main.py", line 1, in <module>  
    print(int(None))  
TypeError: int() argument must be a  
string, a bytes-like object or a number,  
not 'NoneType'
```



int(None) → **raises TypeError** (None is not numeric)

None to Boolean Conversion

Python

```
print(bool(None))
```

Output

False



bool(None) → returns False

Null to Integer / Boolean Conversion

Javascript

```
console.log(Number(null));  
console.log(Boolean(null));
```

Output

```
0  
false
```



bool(None) → returns False

Invalid Number Conversion - Python

```
print(int("hello"))
```

Python

Output

```
Traceback (most recent call last):  
  File "main.py", line 1, in <module>  
    print(int("hello"))  
ValueError: invalid literal for int()  
with base 10: 'hello'
```



invalid numeric conversion → raises ValueError

Invalid Number Conversion - Python

```
print(float("abc"))
```

Python

Output

```
Traceback (most recent call last):  
  File "main.py", line 1, in <module>  
    print(float("abc"))  
ValueError: could not convert string  
to float: 'abc'
```



invalid numeric conversion → raises ValueError

Invalid Number Conversion - JavaScript

JavaScript

```
console.log(Number("hello"));  
console.log(parseInt("abc"));
```

Output

```
NaN  
NaN
```



invalid numeric conversion → **returns NaN** (no error)

Working with Strings

String Repetition



Python

```
a = "*" + "*" + "*"
print(a)
```

Output

```
***
```

Working with Strings

String Repetition

What if we want to print 10 stars?



String Repetition



Python

```
a = "*" * 10  
print(a)
```

Output

```
*****
```

- ✓ String will be **repeated** 10 times
- ✓ This is called **String Repetition**

String Repetition Example



Python

```
s = "Python"  
s = ("* " * 3) + s + (" * " * 3)  
print(s)
```

Output

```
* * * Python * * *
```

Key Differences



Language	Method for String Repetition	Code Example	Result
JavaScript	.repeat() method	<code>str.repeat(3)</code>	"HelloHelloHello "
Python	* operator	<code>str * 3</code>	"HelloHelloHello "

Length of String

Python

```
username = "Ravi"  
length = len(username)  
print(length)
```

Output

4

- ✓ **len()** returns number of **characters** in a **given string**
- ✓ The **return value** is of **data type int**

Python vs JavaScript

Language	Method to Find Length	Code Example	Output
JavaScript	<code>.length</code>	<pre>let str = "Python" ; console.log(str.length);</pre>	6
Python	<code>len()</code>	<pre>str = "Python" console.log(len(str))</pre>	6

Accessing Characters in String

username

"Ravi"

0 1 2 3

username[0]

"R"

- ✓ **Individual** characters in a string can be accessed using their positions (starting from 0)
- ✓ These **positions** are also called as **index**

Accessing Characters in String

Python

```
username = "Ravi"  
first_letter = username[0]  
print(first_letter)
```

Output

R

username

"Ravi"

0 1 2 3

Difference



- ✓ String indexing works the **same way in JavaScript and Python**
- ✓ Indexing starts from **0**
- ✓ Strings are **immutable** (characters **cannot** be **modified** directly)

Accessing Characters in String

Changing a Character

Python

```
message = "sea you soon"  
message[2] = "e"  
print(message)
```

Can we modify a
string like this?



Changing a Character

Python

```
message = "sea you soon"  
message[2] = "e"  
print(message)
```

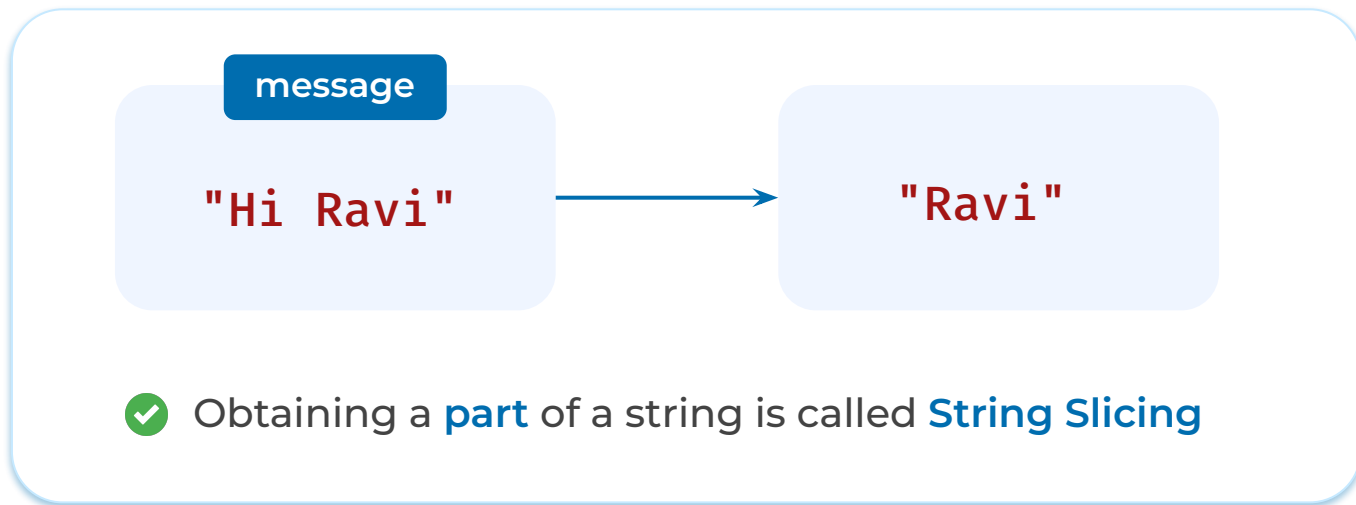
Output

```
TypeError:  
'str' object does not  
support item assignment
```

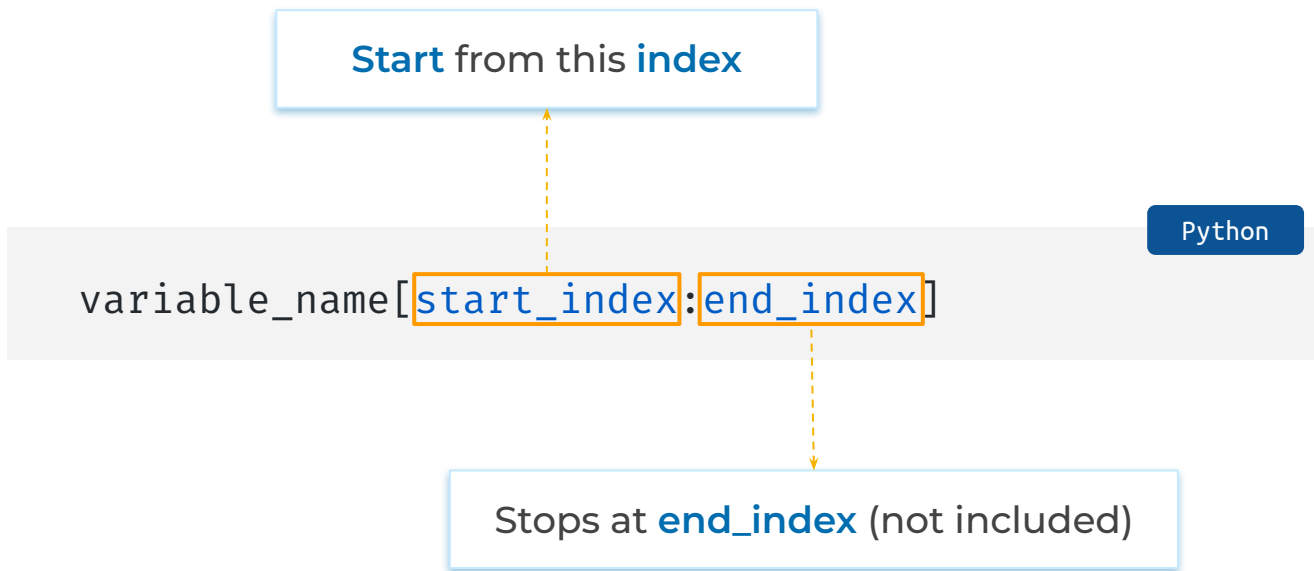
Strings are Immutable

(Can't be modified)

Obtain Part of a String



Slicing Syntax



Working with Strings

Slicing

Python

```
message = "Hi Ravi"  
part = message[3:6]  
print(part)
```

Output

Rav



Slicing

Python

```
message = "Hi Ravi"  
part = message[3:]  
print(part)
```

Output

Ravi

If **end index** is not specified, **slicing** stops at the **end of** the **string**

Slicing

Python

```
message = "Hi Ravi"  
part = message[:2]  
print(part)
```

Output

Hi

If **start index** is not specified, **slicing** starts from the **index 0**

Slicing

Python

```
message = "Hi Ravi"  
part = message[:]  
print(part)
```

Output

Hi Ravi

If **start** and **end index** are not specified, the **sliced** string is same as the **original string**

JavaScript vs Python

Topic	Python	JavaScript	Difference / Notes
Extract part of string	<code>str[start:end]</code>	<code>str.slice(start, end)</code>	Both extract substring; end index excluded
Start index only	<code>str[start:]</code>	<code>str.slice(start)</code>	Extracts till end of string in both
End index only	<code>str[:end]</code>	<code>str.slice(0, end)</code>	Python allows extracting from start up to end; JS needs <code>slice(0, end)</code>
Extract single char	<code>str[index]</code>	<code>str[index]</code> or <code>str.charAt(index)</code>	Similar syntax, both provide the char

Taking Input From User

Taking Input From User



- ✓ **input() function** allows **users** to **enter data**
- ✓ **Data entered** is always stored as a **string**
- ✓ Even if the **input** is a **number** or **boolean**, it will be treated as a **string**



Taking Input From User



When we **download software**, we usually **don't change** the **instructions** in it



But, we **interact** with the **software** through an **interface**, providing **input** that **influences** its **behavior**

Take Input From User



`input()`

- ✓ Allows to take the **input** from the **user**



```
username = input()
```

Example - Python

Python

```
username = input()  
age = input()  
print(username + " is " + age + " years old")
```

Input

```
Ravi  
10
```

Output

```
Ravi is 10 years old
```


Example - JavaScript

Javascript

```
let username = prompt();  
let age = prompt();  
console.log(username + " is " + age + " years old");
```

Input

```
Ravi  
10
```

Output

```
Ravi is 10 years old
```

Take Input From User



- ✓ **input()** function **reads** all input as **strings**
- ✓ Even if a **number** (e.g., 10) is entered, it's treated as a string
- ✓ **Concatenating** strings results in "Ravi is 10 years old"

Key Difference



Python

input() is used to get
user input



JavaScript

prompt() is used
for user input

User input is **always** treated as a **string**, regardless of **data type**

Python vs JavaScript



print() function is used to **display output** to the **console**

Python

```
print("Hello, World!")
```



console.log() function is used to **print output** to the **console**

JavaScript

```
console.log("Hello, World!")
```

print() - Using Comma

Python

```
a = 3  
b = 2  
print(a, b)
```

print() function allows us to **print multiple values** by **passing several variables** as **arguments, separated by commas**

Output

3 2

print()-Using Comma

Using Comma vs Using “+” Operator

Python

```
name = "Rahul"  
age = 25  
print("My name is", name, "and I am", age, "years old.")
```

or

Python

```
name = "Rahul"  
age = 25  
print("My name is " + name + " and I am " + str(age) + " years old.")
```

Using Comma vs Using “+” Operator

Output

Output

```
My name is Rahul and I am 25 years old.
```



Using Comma vs Using “+” Operator

Key Differences

Feature	+	,
Works only with strings?	Yes	No
Auto converts numbers?	No	Yes
Adds space automatically?	No	Yes
Causes error with int?	Yes	No

Quiz Time!

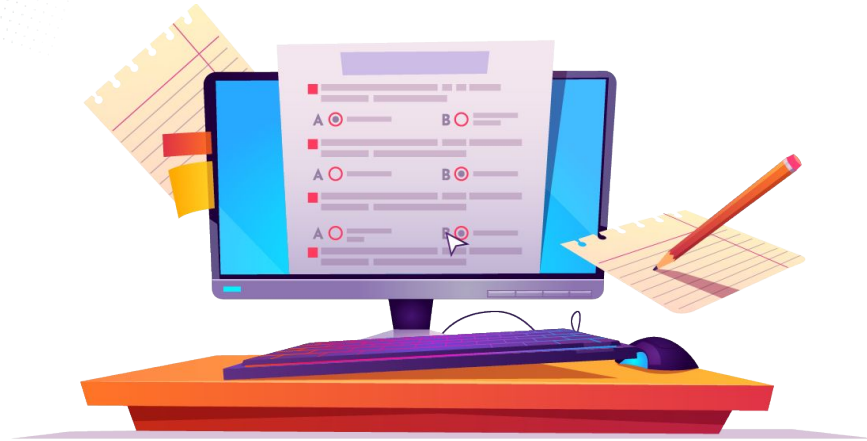


Key Takeaways

- ✓ Working with Strings
- ✓ Type Conversion
- ✓ Input Output Basics



Practice





Next Session

How to Debug Your Code



THANK YOU





ALL THE BEST

